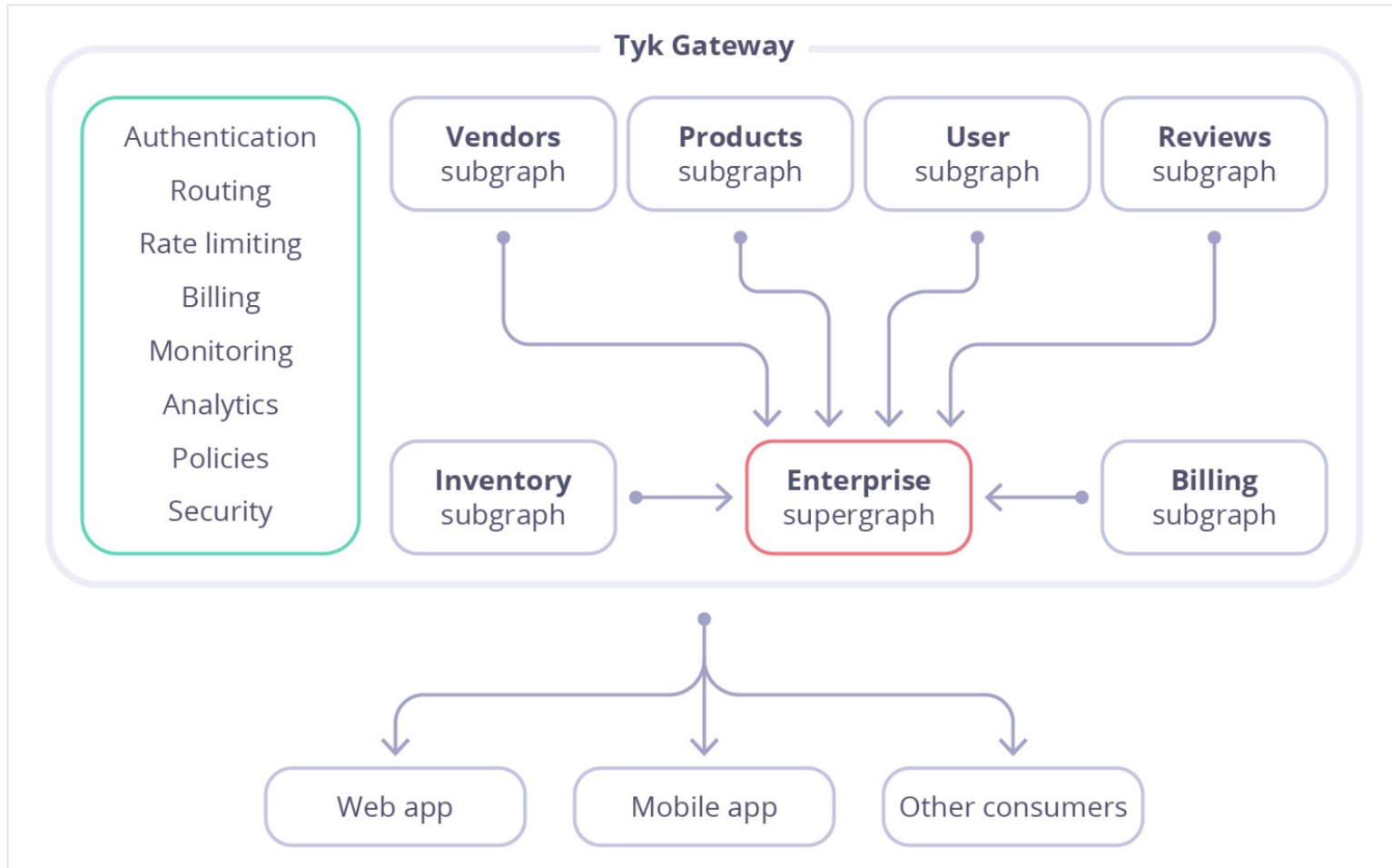


Module 2

# GraphQL Federation

# What is GraphQL Federation

- GraphQL Federation solves the complexity of scaling GraphQL in large enterprises.
- Allows you to break down monolithic GraphQL services into multiple backend services.
- Tyk (v4.0+) supports federation to expose multiple services as a single, unified graph.
- Simplifies maintenance and encourages service ownership.



# Subgraphs and Supergraphs

- Subgraph is a representation of a back-end service and defines a distinct GraphQL schema. It can be queried directly as a separate service or it can be federated into a larger schema of a supergraph.
- Supergraph is a composition of several subgraphs that allows the execution of a query across multiple services in the backend.

# Subgraph Examples

## Users

```
extend type Query {  
  me: User  
}  
  
type User  
@key(fields: "id") {  
  id: ID!  
  username: String!  
}
```

## Products

```
extend type Query {  
  topProducts(first: Int = 5): [Product]  
}  
  
extend type Subscription {  
  updatedPrice: Product!  
  updateProductPrice(upc: String!): Product!  
  stock: [Product!]  
}  
  
type Product @key(fields: "upc") {  
  upc: String!  
  name: String!  
  price: Int!  
  inStock: Int!  
}
```

# Subgraph Examples

## Reviews

```
type Review {  
  body: String!  
  author: User! @provides(fields: "username")  
  product: Product!  
}
```

```
extend type User @key(fields: "id") {  
  id: ID! @external  
  username: String! @external  
  reviews: [Review]  
}
```

```
extend type Product @key(fields: "upc") {  
  upc: String! @external  
  reviews: [Review]  
}
```

## After federating all subgraphs in Tyk, the unified supergraph schema looks like:

```
type Query {  
  topProducts(first: Int = 5): [Product]  
  me: User  
}  
  
type Subscription {  
  updatedPrice: Product!  
  updateProductPrice(upc: String!): Product!  
  stock: [Product!]  
}  
  
type Review {  
  body: String!  
  author: User!  
  product: Product!  
}  
  
type Product {  
  upc: String!  
  name: String!  
  price: Int!  
  inStock: Int!  
  reviews: [Review]  
}  
  
type User {  
  id: ID!  
  username: String!  
  reviews: [Review]  
}
```

# Subgraph and Supergraphs

- Split your system across multiple subgraphs for modularity
- Use `@key`, `@external`, and `@provides` to link types across services
- All subgraphs unified and governed via Tyk Gateway
- Benefits: Faster development, clear service ownership, centralized security
- One supergraph = One entry point for all consumers

# Creating a Subgraph via the Dashboard UI

- Log in to Dashboard → APIs > Add New API > Federation > Subgraph
- Choose a name and provide an upstream URL
- If upstream is protected, select "Upstream Protected" and provide auth details (Header or Certificate)

### Overview

Name

Type

☐ Rest

☐ TCP

☐ GraphQL

☐ UDG

☒ Federation

### Federation

☒ Subgraph

☐ Supergraph  
0 Subgraphs available

### Details

Upstream URL

☐ Upstream Protected

### Details

Upstream URL

☒ Upstream Protected

Upstream protected by

☒ Headers

☐ Certificate

☐ Persist headers for future use

Header name

Header value

# Creating a Subgraph via the Dashboard UI

- Log in to Dashboard → APIs > Add New API > Federation > Subgraph
- Choose a name and provide an upstream URL
- If upstream is protected, select "Upstream Protected" and provide auth details (Header or Certificate)

### Overview

Name

Type

☐ Rest

☐ TCP

☐ GraphQL

☐ UDG

☒ Federation

### Federation

☒ Subgraph

☐ Supergraph  
0 Subgraphs available

### Details

Upstream URL

☐ Upstream Protected

### Details

Upstream URL

☒ Upstream Protected

Upstream protected by

☒ Headers

☐ Certificate

☐ Persist headers for future use

Header name

Header value

# Creating a Supergraph via the Dashboard UI

- Go to APIs > Add New API > Federation > Supergraph
  - In Details, select subgraphs to include
  - Configure supergraph settings like any other API
  - Save → supergraph is available in the API list

### Overview

Name

Type

☐  Rest

☐  TCP

☐  GraphQL

☐  UDG

☒  Federation

### Federation

☐  Subgraph

☒  Supergraph  
3 Subgraphs available

### Details

Subgraphs

### Details

Subgraphs

☒ subgraph-reviews

☒ subgraph-products

☒ subgraph-users

▼

All sub graphs

☒ subgraph-reviews

☒ subgraph-products

☒ subgraph-users

# Defining Global Headers for Supergraphs

- Open supergraph API in Dashboard
  - Go to Subgraphs tab → Global Headers
  - Add header name and value
  - Click Update, then Update API to save changes
  - Headers are forwarded to all relevant subgraphs

super\_fed

VIEW RAW DEFINITION

OPTIONS ▾

UPDATE

Core Settings Subgraphs Schema Playground Advanced Options

▾ Global Headers

Subgraphs

ADD SUBGRAPH

▾ products\_fed



▾ accounts\_fed



▾ reviews\_fed



# Defining Global Headers for Supergraphs

- Open supergraph API in Dashboard
  - Go to Subgraphs tab → Global Headers
  - Add header name and value
  - Click Update, then Update API to save changes
  - Headers are forwarded to all relevant subgraphs

super\_fed

[VIEW RAW DEFINITION](#)

[OPTIONS](#) ▾

[UPDATE](#)

[Core Settings](#) [Subgraphs](#) [Schema](#) [Playground](#) [Advanced Options](#)

▾ Global Headers

Subgraphs

[ADD SUBGRAPH](#)

▾ products\_fed



▾ accounts\_fed



▾ reviews\_fed



# Entities – Defining the Base Entity

- Base entity must have @key directive with fields that uniquely identify it
- Multiple primary keys allowed
- Example:

```
type MyEntity @key(fields: "id") @key(fields: "name") {  
  id: ID!  
  name: String!  
}
```

# Entities – Extending Entities

- Extensions add fields to base entities in other subgraphs
- Must use @key with the same primary key fields
- Primary keys referenced with @external directive
- Example:

```
extend type MyEntity @key(fields: "id") {  
  id: ID! @external  
  newField: String!  
}
```

# Entities – Entity Stubs

- Stubs reference an entity without adding fields
- Minimal info needed to identify the entity
- Example:

```
extend type MyEntity @key(fields: "id") {  
  id: ID! @external  
}
```

# Shared Types & Extending Shared Types

- Shared types have the same name and structure in multiple subgraphs
- Extensions allowed only if normalized resolutions are identical
- Example enums with extensions:

## Subgraph 1

```
enum Example { A, B }  
extend enum Example { C }
```

## Subgraph 2:

```
enum Example { A, B, C }
```

# Extension Orphans

- Orphans occur when extensions are unresolved in federation
- Happens if base type is missing or defined in multiple subgraphs
- Example error: extending a type defined in zero or multiple subgraphs

```
extend type Person {  
  name: String!  
}
```

- Federation fails with an error